Dispensa Operativa Studenti & Quaderno di Laboratorio

Modulo: Git, Unsloth Studio e OpenCode per la Digitalizzazione Industriale

Fondazione ITS MAKER – Bologna

Studente/Gruppo: _____ **Data:** _____ **Postazione VM:** _____

PARTE 0 – Il Cruscotto da Banco (I 6 Comandi Salvavita)

Tieni questa pagina sempre aperta di fianco al terminale durante tutto il laboratorio.

Operazione in Officina Digitale	Comando da Digitare nel Terminale	Quando si usa
0. Configura il tuo cartellino (Solo la 1ª volta)	`git config --global user.name "Tuo Nome"` `git config --global user.email "tuo@itsmaker.it"`	Appena accendi la VM prima di iniziare il laboratorio
1. Preleva un progetto dal Magazzino	`git clone <indirizzo-o-cartella>` `cd <nome-cartella>`	Per scaricare il template di partenza sulla tua VM
2. Ispeziona il banco di lavoro	`git status`	Per vedere quali file sono stati creati o modificati
3. Metti in Cassaforte (Salva Partita)	`git add .` `git commit -m "v1: descrizione modifica"`	**SUBITO DOPO** aver verificato nel browser che la versione funziona!
4. FUNGO DI EMERGENZA (Annulla danni)	`git restore .`	Quando l'AI o una modifica errata rompe il file e vuoi tornare all'ultimo salvataggio buono
5. Avvia il Banco Prova Motori AI	`unsloth studio -p 8888`	Poi apri il browser all'indirizzo `http://localhost:8888`
6. Avvia l'Operatore di Vibe Coding	`unsloth start opencode`	Dentro la cartella del tuo progetto per far scrivere/modificare i file all'AI

LABORATORIO 1 (Ora 1) – L'Ecosistema Open Source e il "Fungo di Emergenza" Git

Esercizio 1A – Ispezione Tecnica di un Repository Open Source (15 min)

Prima di scaricare qualsiasi software industriale Open Source, un tecnico controlla la sua "scheda macchina" (`README.md`).

Apri nel browser i repository indicati dal docente e compila la griglia di collaudo:

Domanda di Verifica	Repository 1 (`unslothai/unsloth`)	Repository 2 (`template-officina`)
1. In una frase, a cosa serve questo progetto?		
2. Quando è stato fatto l'ultimo salvataggio (Commit)?		
3. Quale licenza d'uso ha? (Es. MIT, Apache 2.0)		
4. Posso usarlo gratis in un'azienda meccanica? (SÌ/NO)		

Esercizio 1B – Simulazione di Guasto e Ripristino con Git (30 min)

Obiettivo: Imparare a salvare una configurazione macchina funzionante e recuperarla in 2 secondi dopo un errore distruttivo.

- ☐ **Passo 1 – Configura il tuo nome su Git:**

Apri il Terminale e incolla (sostituendo il tuo nome):

```
git config --global user.name "Nome Cognome"
git config --global user.email "studente@itsmaker.it"
```

- ☐ **Passo 2 – Crea o clona la cartella di collaudo `officina-base`:**

```
mkdir -p ~/officina-base
cd ~/officina-base
git init
```

- ☐ **Passo 3 – Crea la scheda parametri macchina (`parametri_cnc.html`):**

Esegui questo comando per creare il file base della macchina utensile:

```
cat << 'EOF' > parametri_cnc.html
<!DOCTYPE html>
<html lang="it">
<head><meta charset="UTF-8"><title>Parametri CNC Mazak-01</title></head>
<body style="font-family: sans-serif; background: #1e293b; color: white; padding: 20px;">
  <h1>Scheda Parametri CNC Mazak-01</h1>
  <p><strong>Responsabile Turno:</strong> DA COMPILARE</p>
  <ul>
    <li>Velocità Mandrino: <strong>3200 RPM</strong></li>
    <li>Avanzamento Lavoro: <strong>800 mm/min</strong></li>
    <li>Pressione Refrigerante: <strong>6.0 bar</strong></li>
```

```
    </ul>
  </body>
  </html>
  EOF
```

- □ **Passo 4 – Personalizza e Verifica nel Browser:**

Apri `parametri_cnc.html` con un editor di testo, scrivi il tuo **Nome e Cognome** al posto di `DA COMPILARE` e porta la velocità mandrino a **`4500 RPM`**. Apricelo nel browser con doppio clic per verificare che si veda bene.

- □ **Passo 5 – Metti in Cassaforte (Primo Commit):**

Torna nel terminale e salva il punto sicuro:

```
  git status
  git add .
  git commit -m "v1: parametri CNC collaudati a 4500 RPM"
```

- □ **Passo 6 – IL DISASTRO CONTROLLATO!**

Immagina che un collega (o un'AI impazzita) sovrascriva per sbaglio il file:

```
  echo "ERRORE DI SISTEMA - FILE CORROTTO @#!!" > parametri_cnc.html
```

Aggiorna il browser (**F5**): **la scheda tecnica è sparita!**

- □ **Passo 7 – Premi il Fungo di Emergenza (`git restore`):**

Guarda cosa è successo e ripristina l'ultimo salvataggio buono:

```
  git status
  git diff
  git restore .
```

Aggiorna il browser (**F5**): la tua scheda a `4500 RPM` è tornata intatta!

Verifica lo storico dei salvataggi con:

```
  git log --oneline
```

LABORATORIO 2 (Ore 2–3) – Il Banco Prova Motori AI su Unsloth Studio

Esercizio 2A – Avvio di Unsloth Studio e Misurazione Risorse Hardware (45 min)

Obiettivo: Scoprire quanto "pesa" un modello AI sulla RAM e sulla CPU della nostra macchina virtuale al variare dei parametri (`0.6B` vs `1.5B` vs `3B`) e della quantizzazione (`GGUF 2-bit` vs `4-bit`).

- □ **Passo 1:** Apri il *Monitor di Sistema / Gestione Attività* della VM per tenere d'occhio l'uso di **CPU (%)** e **RAM (MB/GB)**.
- □ **Passo 2:** Nel terminale avvia Unsloth Studio:

```
unsloth studio -p 8888
```

- ☐ **Passo 3:** Apri il browser su `http://localhost:8888`.
- ☐ **Passo 4:** Carica uno alla volta i modelli presenti in cache, invia una domanda semplice (*"Elenca 3 controlli di sicurezza prima di avviare un tornio parallelo"*) e compila la **Tabella di Collaudo al Banco**:

Modello GGUF Testato	Livello Quantizzazione	Badge Unsloth (`RECOMMENDED` / `TIGHT` / `OOM`)	RAM Occupata sulla VM (MB/GB)	Velocità Misurata (token/s)	Qualità dell'Italiano (1–5)
`Qwen3-0.6B-GGUF`	`Q4_K_M` (4-bit)				
`Llama-3.2-1B-Instruct-GGUF`	`UD-IQ2_M` (~2-bit)				
`Qwen2.5-Coder-1.5B-Instruct-GGUF`	`Q4_K_M` (4-bit)				
`Qwen2.5-Coder-3B-Instruct-GGUF`	`UD-Q3_K_M` (3-bit)				

Esercizio 2B – Stress Test Industriale nella "Model Arena" (50 min)

Obiettivo: Mettere a confronto diretto (**Model Arena** di Unsloth Studio) un modello ultra-compresso (`1B a 2-bit`) contro un modello bilanciato (`1.5B a 4-bit`) su 3 casi reali di fabbrica per scovare le **allucinazioni**.

Apri la sezione **Model Arena** in Unsloth Studio, seleziona a sinistra il modello più piccolo/compresso e a destra `Qwen2.5-Coder-1.5B-Instruct-GGUF (Q4_K_M)`, poi esegui i 3 test:

TEST 1 – Analisi Rapporto di Fermo Macchina e Calcolo Matematico

Incolla questo prompt:

```
Sei l'assistente del reparto manutenzione. Leggi questa nota operatore:

"Ore 14:20 - Pressa P03 ferma per sovratemperatura olio idraulico (misurati 78°C, limite massimo 65°C). Sostituito filtro intasato cod. FLT-992, fermo macchina totale 45 minuti."

Estrai una tabella con: Macchina, Causa Guasto, Temperatura Rilevata, Delta superamento limite (°C), Ricambio Usato, Minuti Fermo.
```

- **Verifica Tecnica:** Il Delta corretto è `78 - 65 = 13°C`.
- Il Modello A ha scritto: `_____ °C` (Corretto? `[ ] SÌ [ ] NO`)
- Il Modello B ha scritto: `_____ °C` (Corretto? `[ ] SÌ [ ] NO`)

TEST 2 – Estrazione Dati da Bolla di Consegna disordinata

Incolla questo prompt:

```
Da questa email disordinata del fornitore estrai SOLO un elenco puntato con Codice Articolo e Quantità Consegnata:

"Ciao Marco, col furgone di stamattina ti abbiamo mandato 120 pezzi della flangia FL-404, poi
```

```
45 cuscinetti SKF-6204 (ne mancavano 5 rispetto all'ordine di 50) e infine 300 guarnizioni OR-112. I bulloni M10 invece arrivano giovedì prossimo."
```

- **Verifica Tecnica (Attenzione alla trappola!):** I cuscinetti consegnati sono **45** (non 50!) e i bulloni M10 **non sono stati consegnati** (0 pezzi).
- Quale modello è caduto nella trappola? _____

TEST 3 – Generazione di Codice HTML Pronto all'Uso

Incolla questo prompt:

```
Scrivi il codice HTML completo di un pulsante rosso grande con scritto "FERMO EMERGENZA LINEA 2" che quando viene cliccato mostra a schermo l'orario esatto del blocco.
```

- **Conclusione del Banco Prova:** Quale modello lasciamo acceso su Unsloth Studio per guidare OpenCode nel prossimo modulo?
- **Scelta consigliata per la VM:** `unsloth/Qwen2.5-Coder-1.5B-Instruct-GGUF (Q4_K_M)` (oppure il Server di Reparto LAN del Docente se la CPU della VM supera il 90%).

LABORATORIO 3 (Ore 4–6) – Costruire uno Strumento di Digitalizzazione in "Vibe Coding" con OpenCode + Git

Passo Preliminare – Creazione del Cantiere di Lavoro e Avvio di OpenCode (15 min)

- □ **1. Crea la cartella del tuo progetto e inizializza Git:**

```
mkdir -p ~/progetto-maker
cd ~/progetto-maker
git init
```

- □ **2. Avvia OpenCode collegato al motore AI:**
- **Modalità Locale (Piano A):** Assicurati che `Qwen2.5-Coder-1.5B-Instruct-GGUF` sia caricato in Unsloth Studio e lancia:

```
unsloth start opencode
```

- **Modalità Server di Reparto LAN Docente (Piano B – se la tua VM è lenta):**

Dentro OpenCode digita `/model` → `Connect provider` → `Custom` e inserisci i parametri scritti alla lavagna dal docente (`http://<IP-DOCENTE>:8888/v1/`).

Scegli 1 dei 5 Progetti di Digitalizzazione Industriale

Segui rigorosamente il ciclo: **Prompt v1 → Collaudo nel Browser → Commit Git → Prompt v2 → Collaudo → Commit Git → Prompt v3 → Collaudo → Commit Git**.

OPZIONE PROGETTO 1: Scheda Manutenzione Preventiva & Checklist 5S (`manutenzione.html`)

- ☐ **FASE v1 – Incolla in OpenCode:**

```
  Crea in un singolo file chiamato manutenzione.html (con CSS e JavaScript inclusi nello stesso file, senza librerie esterne) una scheda digitale di Manutenzione Giornaliera per un Centro di Lavoro CNC "Mazak-04".

  Inserisci:

  - Campi in alto per: Nome Operatore, Turno (Mattina/Pomeriggio/Notte) e Data odierna automatica.

  - Una lista di 6 controlli pratici (Livello olio guide, Pressione pneumatica 6 bar, Pulizia vasca trucioli, Integrità fungo emergenza, Controllo refrigerante, Pulizia fotocellule) con pulsanti grandi "OK" (verde), "ANOMALIA" (rosso) e campo note per ogni voce.

  - Usa uno stile scuro industriale ad alto contrasto leggibile su monitor di reparto.
```

- ☐ **Collaudo & Salvataggio v1:** Apri `manutenzione.html` nel browser. Se funziona, apri un secondo terminale nella cartella `~/progetto-maker` ed esegui:

```
  git add manutenzione.html

  git commit -m "v1: scheda manutenzione base CNC funzionante"
```

- ☐ **FASE v2 – Incolla in OpenCode:**

```
  Modifica manutenzione.html aggiungendo:

  1. Una barra di avanzamento in alto che mostra la percentuale di controlli completati.

  2. Un avviso lampeggiante rosso in cima alla pagina se almeno un controllo è segnato come "ANOMALIA" ("ATTENZIONE: MACCHINA FERMA - CHIAMARE MANUTENZIONE").

  3. Un pulsante "Salva ed Esporta CSV Fine Turno" che salva i dati nel localStorage del browser (così se ricarico la pagina non perdo le spunte) e scarica un file .csv con l'esito dei controlli.
```

- ☐ **Collaudo & Salvataggio v2:** Premi F5 nel browser, prova i pulsanti e salva su Git:

```
  git add manutenzione.html

  git commit -m "v2: aggiunta barra avanzamento, allarme anomalia ed export CSV"
```

- ☐ **FASE v3 – Incolla in OpenCode:**

```
  C'è un problema operativo: se un operatore clicca "Salva ed Esporta CSV Fine Turno" senza aver inserito il Nome Operatore o senza aver controllato tutte le 6 voci, il file viene scaricato comunque incompleto.

  Aggiungi una validazione che blocca l'esportazione, evidenzia in giallo i campi mancanti e mostra un messaggio chiaro con l'elenco dei controlli ancora da eseguire. Aggiungi anche un pulsante "Inizia Nuovo Turno" con conferma di sicurezza per azzerare la scheda.
```

- ☐ **Collaudo & Salvataggio v3** *(Se l'AI ha rotto il file, usa `git restore .` e riprova!)*:

```
  git add manutenzione.html
```

```
git commit -m "v3: validazione campi obbligatori e reset fine turno"
```

OPZIONE PROGETTO 2: Validatore e Convertitore Distinta Base Ufficio Tecnico (`bom_inspector.html`)

- □ **FASE v1 – Incolla in OpenCode:**

```
Crea un singolo file bom_inspector.html (HTML, CSS e JS tutto incluso, zero dipendenze esterne) per l'Ufficio Tecnico di un'azienda meccanica.

La pagina deve avere:

- Un'area di testo dove incollare una Distinta Base in formato CSV (colonne: Codice_Articolo, Descrizione, Materiale, Quantita, Costo_Unitario).

- Un pulsante "Carica Distinta di Prova (con errori)" che riempie automaticamente l'area con 10 componenti meccanici (carpenteria, viti M8, cuscinetti SKF, alberi rettificati) includendo di proposito 2 codici duplicati, 1 quantità uguale a 0 e 1 riga senza materiale.

- Un pulsante "Analizza Distinta" che mostra i dati in una tabella ordinata.
```

- □ **Collaudo & Salvataggio v1:**

```
git add bom_inspector.html

git commit -m "v1: lettore distinta base CSV con dati di prova"
```

- □ **FASE v2 – Incolla in OpenCode:**

```
Aggiorna bom_inspector.html per aggiungere il controllo qualità automatico della distinta base:

- Evidenzia in rosso nella tabella le righe con Quantita <= 0 o Materiale vuoto.

- Evidenzia in arancione le righe che hanno lo stesso Codice_Articolo ripetuto più volte.

- Mostra in alto 3 riquadri riepilogativi (KPI): Numero Totale Componenti, Numero Errori Trovati, Costo Totale Commessa (somma di Quantita * Costo_Unitario).
```

- □ **Collaudo & Salvataggio v2:**

```
git add bom_inspector.html

git commit -m "v2: validazione errori distinta e calcolo costo commessa"
```

- □ **FASE v3 – Incolla in OpenCode:**

```
Quando ci sono due righe con lo stesso Codice_Articolo, l'ufficio acquisti vuole unificarle sommando le quantità invece di ordinarle due volte, e c'è un bug se il Costo_Unitario usa la virgola invece del punto decimale (es. "12,50" dà NaN).

1. Correggi il calcolo in modo che accetti sia la virgola sia il punto nei prezzi.

2. Aggiungi un pulsante "Ripara Automaticamente Distinta" che unisce i codici duplicati sommando le quantità, rimuove le righe con quantità 0 e permette di scaricare il CSV pulito.
```

- □ **Collaudo & Salvataggio v3:**

```
git add bom_inspector.html

git commit -m "v3: fix virgola decimale e unione automatica codici duplicati"
```

OPZIONE PROGETTO 3: Dashboard Controllo Scarti & Semaforo Qualità (`controllo_scarti.html`)

- **□ FASE v1 – Incolla in OpenCode:**

```
Crea in un singolo file controllo_scarti.html (HTML+CSS+JS autonomo) un contatore digitale per il controllo qualità a bordo macchina.

Inserisci:

- Selezione Codice Lotto e Obiettivo Pezzi del turno (es. 500 pezzi).

- Un pulsante grande verde "+1 PEZZO CONFORME (+10 con tasto rapido)" e 4 pulsanti grandi rossi per registrare gli scarti secondo la causale: "Fuori Tolleranza", "Difetto Superficiale", "Bava di Lavorazione", "Danno Materiale Grezzo".

- Tre contatori giganti: Pezzi Buoni, Pezzi Scartati, Percentuale di Scarto (%).
```

- **□ Collaudo & Salvataggio v1:**

```
git add controllo_scarti.html

git commit -m "v1: contatore pezzi conformi e scarti per causale"
```

- **□ FASE v2 – Incolla in OpenCode:**

```
Arricchisci controllo_scarti.html con:

1. Un semaforo industriale in alto che cambia colore: VERDE se lo scarto è sotto il 3%, GIALLO tra 3% e 5%, ROSSO lampeggiante sopra il 5%.

2. Un grafico a barre orizzontali realizzato in puro CSS/JS (senza librerie esterne) che ordina le 4 causali di scarto dalla più frequente alla meno frequente (analisi di Pareto), così il capoturno vede subito il problema principale.
```

- **□ Collaudo & Salvataggio v2:**

```
git add controllo_scarti.html

git commit -m "v2: semaforo soglie scarto e grafico Pareto causali"
```

- **□ FASE v3 – Incolla in OpenCode:**

```
Problema pratico in reparto: con i guanti da lavoro l'operatore spesso preme per sbaglio due volte un pulsante di scarto e falsa la percentuale.

Aggiungi:

1. Un pulsante "Annulla Ultima Registrazione (Undo)" che cancella l'ultimo inserimento fatto e ricalcola subito percentuale, semaforo e grafico.

2. Un registro in basso delle ultime 10 pressioni con l'orario esatto (HH:MM:SS).
```

- **□ Collaudo & Salvataggio v3:**

```
git add controllo_scarti.html

git commit -m "v3: tasto annulla ultimo inserimento e log orario"
```

OPZIONE PROGETTO 4: Calcolatore Tempi Ciclo, Takt Time e Collo di Bottiglia (`tempi_ciclo.html`)

- **□ FASE v1 – Incolla in OpenCode:**

```
Crea in un singolo file tempi_ciclo.html (HTML+CSS+JS senza librerie esterne) un Calcolatore di Takt Time e Bilanciamento Linea per una linea di assemblaggio meccatronico a 4 stazioni (Stazione 1: Pre-montaggio, Stazione 2: Cablaggio, Stazione 3: Collaudo Elettrico, Stazione 4: Imballaggio).

Inserisci:

- Input per: Minuti lavorativi per turno (default 450 min) e Pezzi richiesti dal cliente per turno (default 300 pezzi).

- Calcolo automatico del Takt Time in secondi per pezzo.

- Input del tempo ciclo (in secondi) per ciascuna delle 4 stazioni e visualizzazione a barre verticali confrontate con la linea rossa del Takt Time.
```

- **□ Collaudo & Salvataggio v1:**

```
git add tempi_ciclo.html

git commit -m "v1: calcolo takt time e confronto 4 stazioni"
```

- **□ FASE v2 – Incolla in OpenCode:**

```
Modifica tempi_ciclo.html aggiungendo:

1. L'identificazione automatica della stazione "COLLO DI BOTTIGLIA" (quella col tempo più alto): colorala in rosso se supera il Takt Time, in arancione se è la più lenta ma sotto il Takt Time.

2. Il calcolo della "Produzione Massima Reale a Fine Turno" basata sul collo di bottiglia e il semaforo "OBIETTIVO CLIENTE RAGGIUNGIBILE: SÌ / NO".
```

- **□ Collaudo & Salvataggio v2:**

```
git add tempi_ciclo.html

git commit -m "v2: rilevamento collo di bottiglia e capacita produttiva reale"
```

- **□ FASE v3 – Incolla in OpenCode:**

```
Aggiungi due funzioni pratiche per il tecnico Tempi e Metodi:

1. Un mini-cronometro (Start / Stop / Invia a Stazione selezionata) direttamente nella pagina per cronometrare dal vivo un'operazione e inserire il valore misurato nella stazione scelta.

2. Correggi il caso in cui l'utente inserisca 0 pezzi richiesti o valori negativi evitando che compaia "Infinity" o "NaN" nei risultati.
```

- **□ Collaudo & Salvataggio v3:**

```
git add tempi_ciclo.html

git commit -m "v3: cronometro integrato e protezione divisione per zero"
```

OPZIONE PROGETTO 5: Generatore Etichette Pallet & Tracciabilità Lotto (`etichette_lotto.html`)

- **□ FASE v1 – Incolla in OpenCode:**

```
Crea un singolo file etichette_lotto.html (HTML+CSS+JS autonomo, zero internet) per generare schede di tracciabilità bancali in officina.

La schermata deve essere divisa in due:

- A sinistra il modulo di inserimento: Codice Articolo, Descrizione Pezzo, Numero Lotto (generabile anche in automatico nel formato LOT-AAAAMMGG-XX), Quantità Pezzi nel cassone, Operatore e Reparto Destinazione (Trattamento Termico, Verniciatura, Montaggio, Spedizione).

- A destra l'anteprima in tempo reale dell'etichetta industriale con bordi neri spessi, caratteri giganti leggibili dal muletto a 3 metri di distanza e un codice a barre SVG generato direttamente in JavaScript a partire dal Numero Lotto.
```

- **□ Collaudo & Salvataggio v1:**

```
git add etichette_lotto.html

git commit -m "v1: generatore etichetta bancale con codice a barre SVG"
```

- **□ FASE v2 – Incolla in OpenCode:**

```
Aggiungi a etichette_lotto.html:

1. Un selettore "Stato Qualità Lotto" con 3 opzioni che cambiano l'intestazione dell'etichetta: "CONFORME - LIBERO PASSAGGIO" (Verde), "IN ATTESA COLLAUDO" (Giallo), "BLOCCATO / NON CONFORME" (Rosso con banda diagonale).

2. Un pulsante "Registra Bancale e Stampa" che aggiunge il bancale a una tabella "Storico Bancali Prodotti Oggi" in fondo alla pagina (salvata in localStorage) e apre la finestra di stampa (@media print deve stampare SOLO l'etichetta a pagina intera, nascondendo il form).
```

- **□ Collaudo & Salvataggio v2:**

```
git add etichette_lotto.html

git commit -m "v2: stato qualita visivo, storico bancali e layout di stampa"
```

- **□ FASE v3 – Incolla in OpenCode:**

```
Nella tabella "Storico Bancali Prodotti Oggi" manca la possibilità di recuperare un'etichetta se si è inceppata la stampante, e il numero progressivo del lotto non avanza da solo dopo ogni registrazione.

1. Fai in modo che cliccando "Registra Bancale" il contatore finale del lotto aumenti automaticamente di 1 (es. da LOT-20261018-01 a LOT-20261018-02).

2. Aggiungi su ogni riga dello storico un pulsante "Ricarica Etichetta" che riporta tutti i dati di quel bancale nell'anteprima per ristamparla.
```

- **□ Collaudo & Salvataggio v3:**

```
git add etichette_lotto.html
git commit -m "v3: incremento automatico lotto e pulsante ristampa dallo storico"
```

Passo Finale (Blocco B6) – Generazione del Manuale `README.md` e Collaudo Incrociato

- **□ 1. Fai scrivere il manuale d'uso a OpenCode:**

```
Crea nella cartella un file README.md ben formattato in italiano per gli operatori di reparto che spieghi:
- Nome dello strumento e problema aziendale risolto.
- Come aprirlo e usarlo passo dopo passo.
- Le funzioni sviluppate nelle versioni v1, v2 e v3.
```

- **□ 2. Salva il manuale su Git e verifica i 4 punti di salvataggio:**

```
git add README.md
git commit -m "docs: aggiunto manuale operativo README.md"
git log --oneline
```

- **□ 3. Scheda di Collaudo Incrociato (Scambia la postazione col compagno di banco per 5 minuti):**
- Nome del compagno collaudatore: _____
- Il file `.html` si apre senza internet? `[ ] SÌ [ ] NO`
- Ho provato a inserire dati vuoti o sbagliati: il programma avvisa senza bloccarsi? `[ ] SÌ [ ] NO`
- `git log --oneline` mostra almeno 4 commit ordinati? `[ ] SÌ [ ] NO`

LABORATORIO ESTENSIONE (Ore 7–8) – Dati Sporchi CSV & File `AGENTS.md`

Esercizio 4A (Ora 7) – Importare un File CSV "Sporco" dall'Ufficio Tecnico

1. Crea nella cartella `~/progetto-maker` un file chiamato `dati_reparto_sporchi.csv` incollando questo comando nel terminale:

```
cat << 'EOF' > dati_reparto_sporchi.csv
Codice_Articolo;Descrizione;Materiale;Quantita;Costo_Unitario
FL-100;Flangia Motore;Acciaio C45;12;18,50
VT-M8x30,Vite Brugola M8x30,Inox A2,200,0.15
FL-100;Flangia Motore;Acciaio C45;8;18,50
CS-6205;Cuscinetto Radiale;;0;9,80
;;
```

```
  AL-440;Albero Trasmissione;39NiCrMo3;-4;42,00
  EOF
```

1. **Osserva i 5 difetti reali dentro `dati_reparto_sporchi.csv`:**

- Righe separate da `;` mescolate a righe separate da `,` (`VT-M8x30`).
- Prezzi con la virgola (`18,50`) e col punto (`0.15`).
- Codice `FL-100` duplicato su due righe.
- Riga `CS-6205` senza materiale e con quantità `0`.
- Riga completamente vuota `;;` e quantità negativa `-4`.

1. **Chiedi a OpenCode di creare o aggiornare il tuo tool per importare e bonificare `dati_reparto_sporchi.csv`** e fai un nuovo `git commit -m "v4: parser CSV sporco multi-separatore"`.

Esercizio 4B (Ora 8) – Creare il Regolamento di Fabbrica `AGENTS.md`

Crea nella cartella `~/progetto-maker` il file `AGENTS.md` che OpenCode leggerà automaticamente prima di ogni modifica:

```
cat << 'EOF' > AGENTS.md
# Standard Tecnico Officina ITS MAKER Bologna
- Usa esclusivamente l'ITALIANO per interfaccia, pulsanti, avvisi e commenti nel codice.
- Non usare mai librerie esterne da internet (nessun link http/https a CDN): il file deve funzionare su PC industriali isolati dalla rete.
- Usa pulsanti grandi (altezza minima 48px) adatti a schermi touch industriali.
- Rispetta i colori semaforici ISO: Verde (#16a34a) = Conforme/OK, Giallo (#eab308) = Attenzione/Manutenzione, Rosso (#dc2626) = Anomalia/Blocco.
EOF
```

- ☐ Chiedi a OpenCode: *"Adegua l'interfaccia del nostro programma alle regole scritte in AGENTS.md aggiungendo in alto il logo testuale 'ITS MAKER BOLOGNA - REPARTO DIGITALIZZATO'"*.
- ☐ Verifica nel browser e salva con `git add . && git commit -m "v5: standardizzazione grafica secondo AGENTS.md"`.

Pronto Soccorso Studenti (Cosa fare se qualcosa si blocca)

Cosa vedi sullo schermo	Perché succede	Cosa fare subito (in 30 secondi)
OpenCode scrive il codice nella chat ma il tuo file `.html` non cambia	Il server Unsloth ha intercettato gli strumenti oppure il modello scelto è troppo piccolo.	Chiudi OpenCode e rilancialo col comando `unsloth start opencode`, oppure copia il codice dalla chat e incollalo nel file `.html`.
Dopo l'ultima modifica di OpenCode la pagina web è bianca	L'AI ha dimenticato di chiudere una parentesi o ha cancellato	**Non toccare nulla a mano!** Vai nel terminale e digita:

Cosa vedi sullo schermo	Perché succede	Cosa fare subito (in 30 secondi)
o i pulsanti non funzionano più	una parte buona del file.	`git restore .` Tornerai subito alla versione precedente funzionante.
Git dice: `Author identity unknown / Please tell me who you are`	Non hai ancora inserito nome ed email del punto 0.	Esegui: `git config --global user.name "Tuo Nome"` `git config --global user.email "tuo@itsmaker.it"`
La macchina virtuale è lentissima mentre l'AI scrive	Tutte le 24 VM stanno usando la CPU insieme.	Ferma il modello locale su Unsloth Studio e chiedi al docente l'indirizzo IP per collegare OpenCode al **Server LAN Docente (Piano B)**.